Lara Hernandez Senior Go Developer

Lisbon, Portugal (Remote) • godev@gmail.com • +351 91 555 0182

PROFILE SUMMARY

- **Senior Go Developer** with **8 years of experience** building **cloud-native Go services at edge scale** across **edge networking, CDN platforms, and developer infrastructure**, specializing in **gRPC microservices, goroutine concurrency tuning, and Kubernetes operator development**.
- Hands-on coverage across **Go** (**Go 1.23**), **HTTP** (**chi**), **persistence** (**pgx**, **PostgreSQL**), **messaging** (**NATS JetStream**), and **containers** (**Kubernetes**, **AWS**) with strong fundamentals in **small interfaces, error values, composition over inheritance, and a working command of generics, context, and channels**.
- Deep expertise in **goroutine and channel concurrency**, **gRPC and protobuf contract design**, **Kubernetes operator development**, and **pprof-driven performance tuning**, applying methodologies such as **context propagation and errgroup orchestration** and **zero-allocation hot paths with escape-analysis discipline** to deliver **reliable, high-throughput Go services fit for global edge scale**.
- Engaged collaborator working **cross-functionally** with **Product, SRE, Security, and Platform teams** in **trunk-based and async-first remote teams**, contributing to API design forums, on-call rotations, and architecture reviews with an ownership-first mindset and clean handoffs.
- Mentor who shares technical excellence and fosters a culture of **service reliability** and **idiomatic Go discipline** through PR reviews and pattern docs, while running **Go guild and API design review sessions** and authoring widely used service and persistence templates.

TECHNICAL SKILLS

Languages & Runtime:	Go 1.23, Go 1.22, Go 1.21, generics, context, channels, sync primitives, errgroup, semaphore, goroutines, error values, iota
HTTP, gRPC & APIs:	chi, gin, echo, fiber, net/http, grpc-go, protobuf, gorilla/mux, REST, OpenAPI, middleware, graceful shutdown
Distributed Systems & Messaging:	NATS JetStream, Kafka, RabbitMQ, etcd, Consul, circuit breakers, retries, idempotency, sagas, outbox
Persistence & Data:	pgx, sqlx, sqlc, GORM, database/sql, PostgreSQL, Redis, DynamoDB, MongoDB, golang-migrate, goose, atlas
Testing, Benchmarking & Quality:	testing, testify, gomock, testcontainers, table-driven tests, fuzz tests, testing.B, go test -race, golangci-lint, staticcheck, gofumpt
Cloud-Native & Kubernetes:	Kubernetes, controller-runtime, kubebuilder, Operator SDK, CRDs, Helm, ArgoCD, Docker, distroless, scratch images
Observability & Performance:	slog, zap, zerolog, Prometheus client_golang, OpenTelemetry, Grafana, Jaeger, pprof, runtime/trace, escape analysis
Build, Modules & CI/CD:	go modules, go build, cross-compilation, Makefiles, GoReleaser, GitHub Actions, GitLab CI, AWS, GCP, static binaries

EDUCATION

Universidade de Lisboa B.S. in Computer Science — Lisbon, Portugal • Sep 2014 - Jun 2018

WORK EXPERIENCE

Cloudflare Senior Go Developer — Lisbon, Portugal (Remote) • May 2021 - Present

- Owned idiomatic Go service development end to end on the **edge access and zero-trust gateway platform** serving **55M req/s peak**, shipping **auth gateway**, **policy engine**, and **audit pipeline** across **18** small-interface Go services held to Effective Go and the Go proverbs.
- Tuned concurrency and goroutines with **errgroup-bounded worker pools with context cancellation**, semaphore-gated fan-out, and sync.Pool reuse on hot allocators, capping worker concurrency at **2048** and pulling steady-state goroutine count under peak from **180k** down to **32k** while closing every leak surfaced by go test -race.
- Built API surfaces with **chi** for REST edges and grpc-go for east-west traffic, leaning on **gRPC for east-west with REST edges via chi**, protobuf contracts, interceptor middleware, and graceful shutdown across **160+** endpoints, dropping p95 from **42ms** to **9ms** on the hot policy-check path.
- Designed back-end and distributed systems around **stateless services with NATS JetStream and etcd leader election**, circuit breakers, retries with jitter, and idempotency keys on every consumer, fanning out across **320+** edge regions and sustaining peak load of **120k msg/s** with zero duplicate-event incidents over the last four quarters.
- Built cloud-native tools and Kubernetes integrations with **controller-runtime operators with kubebuilder CRDs**, Helm chart packaging, and ArgoCD-driven GitOps, owning **11** production CRDs that reconcile state across **42** clusters with zero manual hotfixes since rollout.
- Wired observability and production operations through **slog structured logs with OpenTelemetry traces and RED metrics**, Prometheus client_golang exporters, health and readiness probes, and a documented runbook per service, dropping MTTR from **48min** to **11min** and cutting on-call page volume by **61%**.
- Shipped builds and deployment with **distroless static binaries with multi-arch cross-compilation**, go modules pinned with vendoring on release branches, and GitHub Actions pipelines on **Kubernetes** releasing **multiple times daily**, shrinking container images from **380MB** down to **18MB** across the fleet.

Synadia (NATS.io) Go Developer

Remote (EU) • Jul 2018 - Apr 2021

- Drove testing, benchmarking, and code quality with **table-driven tests with testify and Testcontainers**, gomock interfaces, testing.B benchmarks, fuzz harnesses on parser code, and golangci-lint plus staticcheck gates, lifting race-detector coverage from **38%** to **86%** and cutting data-race incidents per release by **73%**.
- Owned data access and persistence with **pgx connection pooling with golang-migrate versioning**, sqlc-generated queries, prepared statements, and Redis-backed caching, dropping query p99 on the account-lookup path by **68%** and shipping **140+** schema migrations with zero failed-rollback incidents.
- Drove performance and profiling work with **pprof CPU and heap profiling with runtime/trace**, escape-analysis review on every PR touching the hot path, and benchstat regression gates, dropping hot-path allocations from **2.4GB/min** to **290MB/min** and tightening GC pause variance enough to retire the steady-state freeze runbook.
- Strengthened idiomatic Go foundations across the team through **small interfaces, error wrapping, and table-driven design**, package layout reviews, and Effective Go reading sessions, while mentoring **6** engineers through pair-programming, code-review checklists, and a written Go idiom guide adopted across two squads.